

Yazılım Bakımı: Değişikliğin Yayılması

Ömer Faruk Yavuz

August 2026

Özet

Bir test, kodun doğruluğunu denetleyen bir araç olmanın ötesinde, sistem hakkında edinilmiş ve kod tabanına sabitlenmiş bir bilgi birimidir. Farklı test türleri farklı türde bilgi taşır: olağan bir test korunması istenen davranışı, regresyon testi geçmişte yaşanmış bir bozulmayı, entegrasyon testi bileşenler arası sözleşmeyi, sınır durumu testi ise sistemin geçerlilik alanının nerede bittiğini kaydeder. Bu gözlem, test kümesini bir kalite güvence mekanizması olmaktan çıkarıp sistemin davranışsal hafızası konumuna taşır.

Bu dört bilgi türünden davranış, yapı, evrim ve sınır olmak üzere dört boyutlu bir çerçeve türetilir. Literatür taraması, bu dört boyutun her birinin ayrı araştırma hatlarında karşılığı bulunduğunu, ancak dördünün tek bir asgari taksonomi olarak birleştirildiği yerleşik bir kuram bulunmadığını göstermektedir. Dolayısıyla çerçeve bir bulgu değil, sınırlanabilir bir hipotezdir. Beşinci boyut adayı olarak amaç öne çıkmakta, ancak ondan yapısal olarak farklı bir aday daha bulunmaktadır: kanıt. İlk dört boyut sistem hakkında ne bilindiğini, kanıt boyutu ise bilinenin nereden bilindiğini konu alır.

Çerçevenin pratik değeri, kendisinden çok terk edildiği noktada ortaya çıkar. Asıl problem bir hatanın giderilmesinden sonra aynı düzeltmenin başka bir yerde yeni hata üretmesidir. Bu olguya karşı işleyen sıra, lokal problemin anlaşılması, küresel etkinin belirlenmesi, seçeneklerin üretilmesi ve en az yan etki üreten seçeneğin seçilmesidir. En küçük kod değişikliği ile en küçük sistem riski aynı şey değildir.

Ölçüm tarafında merkezî bulgu, kapsamın anlamayla eşitlenemeyeceğidir. Tam kapsama sahip bir kodun neden öyle yazıldığı bilinmiyor olabilir. Bu nedenle hedef, çalıştırılan kod oranını yükseltmek değil sistem hakkındaki belirsizliği azaltmaktır.

Giriş

Yazılım geliştirmede bir hatanın giderilmesi, o hatanın tekrar ortaya çıkmayacağı anlamına gelmez. İki ayrı olgu birbirinden ayrılmalıdır: aynı hatanın tekrar etmesi ve giderilen hatanın yerine yenisinin doğması. Birincisi için yerleşik bir mekanizma vardır; ikincisi ise daha zor bir problemdir, çünkü yeni hatanın nerede ortaya çıkacağı önceden bilinmemektedir.

Notun izlediği yol, bu problemin bir benzetmeden yola çıkılarak kavram-sallaştırılması ve ardından benzetmenin bırakılıp problemin doğrudan ele alınmasıdır. Bu iki aşama, notta ayrı ayrı kaydedilmektedir; ikincisi birincinin sonucu olmadığı gibi, birincinin geçersizliğini de göstermez.

Merkezî terimler burada tanımlanır. **Regresyon**, daha önce doğru çalışan bir davranışın bir değişiklik sonrası bozulmasıdır. **Değişmez** (*invariant*), programın belirli bir noktasında her zaman doğru olması gereken koşuldur. **Dalga etkisi** (*ripple effect*), bir değişikliğin doğrudan dokunulmayan kod parçalarında yarattığı davranış değişikliğidir. **Lokal optimizasyon**, bir problemin yalnızca ortaya çıktığı bölge dikkate alınarak çözülmesidir; **küresel optimizasyon** ise çözümün sistem genelinde bıraktığı durumun dikkate alınmasıdır.

Yaygın ve düzeltilmesi gereken kabul şudur: bir düzeltmenin güvenliği, değiştirdiği satır sayısı ile ters orantılı sayılır. Notta gösterildiği üzere iki büyüklük birbirinden bağımsızdır; iki satırlık bir değişiklik ortak bir sözleşmeyi bozarak geniş bir etki üretebilirken, sekiz satırlık bir değişiklik davranışı yalıtarak etkisini sınırlayabilir.

Notun kapsamı, sistem hakkındaki bilginin kod tabanında nasıl biriktiği, bu birikimin nasıl ölçülebileceği ve bir değişikliğin etkisinin nasıl öngörülebileceğiyle sınırlıdır. Belirli araçlar, diller ve kütüphaneler kapsam dışıdır.

Beyaz Sayfa Benzetmesi

Benzetmenin ilk hali şöyledir: yazılım beyaz bir sayfa olarak düşünülür, kod bu sayfaya yazılan şeydir, testler ise kenarlara alınan notlar ve karalamalardır. Sayfa karardıkça yazılımın iyileştiği öne sürülür.

Benzetmenin öğeleri şu karşılıkları verir:

- Kod, sayfaya yazılan asıl metindir.
- Testler, metnin kenarına alınan notlardır.
- Hatalar, yanlış yazılmış ya da fark edilmemiş yerlerdir.
- Yeniden düzenleme, yazıyı silip daha iyi biçimde tekrar yazmaktır.
- Kapsam, sayfanın ne kadarının fiilen denetlendiğidir.
- Sürekli tümleştirme, her yeni yazıdan sonra sayfanın otomatik olarak denetlenmesidir.
- Kullanıma verilen sürüm, okunacak temiz nüshadır.

Benzetmenin taşıdığı asıl fikir, iyi yazılımın üzerinde hiç iz bulunmayan yazılım olmadığıdır. Üzerinde düşünmenin izleri bulunan yazılım iyi yazılımdır. Testlerin çokluğu ilk bakışta sistemi karmaşıktırıyor gibi görünür; oysa o testler sistem hakkında bilinenlerin temsilidir.

Bu okumada karanlık olan şey kodun kendisi değil, sistem hakkında biriktirilen bilgidir. Buradan bir görünürdeki çelişki doğar: sayfa karardıkça sistem daha temiz hale gelir. Çelişki, iki farklı şeyin aynı sözcükle adlandırılmasından kaynaklanır; kararan şey kod karmaşıklığı değil bilgi yoğunluğudur.

Benzetmenin bir uzantısı, yeniden düzenlemenin ne olduğunu da açıklar. Yeniden düzenleme sayfayı tekrar beyazlatmak değildir; karalamaların içindeki gereksiz gürültünün temizlenip elde edilen bilginin daha iyi bir biçime sokulmasıdır. Beyaz sayfa bilinmeyen sisteme, karalanmış sayfa anlaşılmış sisteme karşılık gelir.

Bu benzetmenin sınırı notun ilerleyen bölümlerinde ele alınmaktadır; burada kaydedilmesi gereken, sayfanın kararmasının tek başına bir kazanım sayılmadığıdır.

Testlerin Taşıdığı Dört Bilgi Türü

Farklı test türleri, aynı biçimde yazılmalarına rağmen farklı sorulara cevap verir. Her testin arkasındaki soru, testin türüne göre değişir.

Dört bakış

Bakış	Temel soru	Testin söylediği
Davranış	Sistem ne yapmalı?	Bu davranış doğrudur ve korunmalıdır
Regresyon	Neyi kaybetmek istemiyoruz?	Burada daha önce bir bozulma yaşandı
Entegrasyon	Parçalar birlikte nasıl davranıyor?	Bu bileşenler arasında şu sözleşme var
Sınır	Sistem nerede bozulmaya başlıyor?	Sistemin sınırı burada

Bu dördünün zaman yönleri de farklıdır. Davranış testi şimdiki zamana bakar ve sistemin mevcut sözleşmesini ifade eder. Regresyon testi geçmişe bakar ve sistemin hafızasını oluşturur; burada test yalnızca doğrulama yapmaz, geçmişte yaşanmış bir olayın bilgisini kod tabanına sabitler. Entegrasyon testi ilişkilere bakar ve sistemin topolojisini anlatır. Sınır durumu testi ise sistemin davranış uzayının geometrisini keşfeder.

Bu okumanın sonucu, test kümesinin zamanla bir bilgi deposuna dönüşmesidir. İyi bir test kümesine bakan bir kişi yalnızca kodun çalıştığını değil, sistemin ne yapması gerektiğini, geçmişte nerelerde bozulduğunu, bileşenlerin nasıl bağlandığını ve sınırlarının nerede olduğunu okuyabilir. Dolayısıyla test kümesini yalnızca bir kalite güvence mekanizması saymak eksik kalır; daha güçlü okuma, test kümesinin sistemin davranışsal hafızası olduğudur.

Aynı sonuç benzetmeye geri döndürülebilir: başlangıçta sistem hakkında bilinenlerin tamamı koddur; zamanla bilinenlerin tamamı kod ile testlerin birleşimi haline gelir.

İlk soyutlama denemesi

Dört bakış, test türü olmaktan çıkarılıp daha temel kavramlar olarak ifade edilmeye çalışıldığında ilk aday şu dördtüdür:

- **Değişmez** (*invariant*) — neyi doğru kabul ediyoruz. Beklenti, kural, sözleşme, tutarlılık. Özü: doğru olan nedir.
- **Hafıza** (*memory*) — neyi yaşadık ve kaybetmek istemiyoruz. Öğrenme, geçmiş hata, tecrübe. Özü: ne öğrendik.
- **İlişki** (*relation*) — şeyler birbirleriyle nasıl davranıyor. Bağlantı, etkileşim, bağımlılık. Özü: neler nasıl bağlanıyor.
- **Sınır** (*boundary*) — nerede çalışmayı bırakıyor ya da davranış değiştiriyor. Limit, tolerans, geçersiz durum. Özü: nereye kadar.

İkinci bir adlandırma denemesi kimlik, tarih, yapı ve sınır dördlüsüdür. Üçüncüsü ise daha felsefi bir düzeyde varlık, oluş, ilişki ve limit biçimindedir.

Bu denemelerin tümü, bu noktada, konuşmayı yürütenlerin kendi türetmeleridir; yerleşik bir taksonomi değildir. Bu ayrımın açıkça kaydedilmesi gerekmektedir, çünkü aynı sözcüklerin literatürde farklı ve daha dar anlamları bulunmaktadır.

Yerleşik terimlere yönelme

Kendi türetmelerden vazgeçilip literatürdeki karşılıklar arandığında tablo şöyledir:

Türetilen kavram	Literatürdeki yakın karşılıklar
Kimlik	identity, purpose, function, state
Tarih	history, evolution, dynamics, process
Yapı	structure, relationships, architecture, organization
Sınır	boundary, system/environment, context, scope

Sistem kuramı literatüründe yerleşik olan ayrımlar bunlardan farklıdır. Genel sistem kuramında sistem, çevre, yapı ve süreç dörtlemesi ile yapı, davranış, işlev ve çevre dörtlemesi kullanılır. Tarih boyutu ise sistem dinamiği, karmaşık uyarlanırlar sistemler ve yazılım evrimi alanlarında doğal bir kavramdır; ancak kimlik, tarih, yapı ve sınır biçiminde standart bir dörtleme bulunmamaktadır.

Bu bulgunun sonucu, ilk gözlemin literatüre zorla tek bir taksonomi olarak eşlenmemesi gerektiğidir.

Yerleşen dörütlü ve üst başlık

Kavramların içeriğı kesinleştirildikten sonra adlandırma seçenekleri şunlardır: davranış, tarih, yapı ve sınır; davranış, evrim, yapı ve sınır; işlev, tarih, yapı ve sınır; sözleşme, evrim, mimari ve sınır.

Tercih edilen dörütlü davranış, evrim, yapı ve sınırdır. Gerekçe, dört kavramın birbirini en az örtmesidir. Tarih yerine evrim kullanılması, geçmişte ne olduğundan çok sistemin zaman içinde nasıl değiştiğini ifade eder.

Üst başlık için değerlendirilen adaylar arasında sistem bilgisi, sistem anlayışı, sistem modeli, sistem betimlemesi, sistem karakterizasyonu, sistem semantığı, sistem özellikleri, sistem anatomisi, sistem dinamiğı, sistem temsili, sistem bağlamı, sistem ontolojisi, sistem perspektifi ve sistem boyutları bulunmaktadır. Bunlar arasındaki anlam farkları belirleyicidir: sistem bilgisi sahip olunan bilgiyi, sistem anlayışı bu bilgiden oluşan zihinsel modeli, sistem modeli bu boyutlar üzerinden kurulan temsili, sistem karakterizasyonu ise karakterize etme işlemini adlandırır.

Seçilen üst başlık sistem anlayışıdır (*system understanding*). Bu ifade İngilizcede kullanılan doğal bir tabirdir; karmaşık sistemlerin kavranması, mühendisin bir sistem hakkındaki zihinsel modeli ve gereksinim çalışmalarında sistem anlayışının geliştirilmesi bağlamlarında geçer. Ancak sistem kuramının resmî bir terimi ya da kabul edilmiş bir çerçeve adı değildir. Dolayısıyla terim mevcuttur, dört boyutlu anlamlandırma ise bu çalışmaya özgüdür. Bu ayrımın korunması, çerçevenin literatürde mevcut bir dörtleme gibi sunulmasını engeller.

Dört Sayısının Sınanması

Dört sayısı, testlerden tümevarımla ulaşılmış bir sonuçtur; matematiksel bir zorunluluk ya da literatürde kabul edilmiş bir sayı değildir. Sayının savunulabilmesi için gösterilmesi gereken şey, bir sistem hakkında anlamlı bir zihinsel model kurulurken birbirinden ayrıştırılabilen dört bilgi türü bulunduğudur.

Bu gösterilmediği sürece hemen ortaya çıkan itirazlar şunlardır: amaç nerededir, çevre nerededir, durum nerededir, bağımlılıklar nerededir. Bunların her birinin dört kategoriden birine yerleştirilebilmesi durumunda dört sayısı güçlenir.

Buradan doğan araştırma sorusu şudur: bir yazılım sistemini anlamak için gereken asgari bağımsız boyut kümesi nedir. Bir boyutun sınanma ölçütü, o boyut çıkarıldığında hangi soruların artık cevaplanamadığıdır.

Literatür taramasının sonucu

Tarama, bu soruya verilmiş yerleşik bir cevap bulunmadığını göstermektedir. Ancak boyutların parçaları literatürün farklı bölgelerinde mevcuttur.

Program anlama alanı doğrudan bu soruyla ilgilenir ve geliştiricinin program hakkında zihinsel model kurması olarak tanımlanır. Bu model koddan başlayıp mimariye ve uygulama alanına kadar farklı soyutlama seviyelerinde bulunabilir. Alanda tek bir asgari boyut kümesi üzerinde uzlaşma yoktur. Farklı bilişsel modeller kullanılmaktadır: yukarıdan aşağı yaklaşımda geliştirici önce programın amacı hakkında bir hipotez kurup ayrıntıya iner (Brooks, 1983); başka modeller programı hedefler, alt hedefler, nesneler, planlar, veri akışı, denetim akışı ve ilişkiler üzerinden ele alır. Şema temelli modellerde çalışma temsili, bilgi tabanı, hedefler ve stratejiler ayrı bileşenler olarak tanımlanır (Détienne, 2002). Yüksek seviyeli bilgi ihtiyaçları özellikle yapı ve işlev olarak ayrılmaktadır.

Sistem mühendisliği tarafında yapı ve davranış açıkça modellemenin amacı olarak tanımlanır. Sistem düşüncesi yaklaşımında mühendisin incelediği unsurlar sınırlar, bağlam ve yaşam döngüsüdür. Model tabanlı sistem mühendisliğinde dört alan gereksinimler ve kabiliyetler, davranış, mimari ve yapı, ile doğrulama ve geçerlemedir. Sistem sınırı, ilgilenilen sistem ile dış varlıklar arasındaki ayrımı yapar; bağlam diyagramı ise sistem ile dış dünya arasındaki girdi, çıktı ve etkileşimleri gösterir (INCOSE, 2015).

Yazılım evrimi alanı başlı başına geniş bir alandır. Sistemin yalnızca mevcut haliyle değil nasıl bu hale geldiğiyle de anlaşılması gerektiği vurgulanır; sürüm geçmişi, değişiklik geçmişi, bakım ve mimari evrim incelenir. Yazılımın yapısına zaman katmanı ekleyen üst modeller geliştirilmiştir (Girba ve Ducasse, 2006).

Üst üste konduğunda tablo şöyledir:

Boyut	Program anlama	Sistem müh.	Yazılım evrimi
Davranış	Güçlü	Güçlü	Mevcut
Yapı	Güçlü	Güçlü (mimari)	Mevcut
Evrin	İlgili	Yaşam döngüsü	Merkezî
Sınır	Bağlam/alan	Çok güçlü	Zayıf

Bu tablodan çıkan sonuç iki yönlüdür. Literatür asgari dört boyut dememektedir; ancak dört boyutun her birinin bağımsız araştırma hattı olarak karşılığı vardır. Dolayısıyla ne bu çerçevenin keşfedildiği ne de literatürde hiç karşılığı bulunmadığı söylenebilir. Doğru konum, çerçevenin sınanabilir bir hipotez olarak kaydedilmesidir.

Beşinci boyut adayları

Amaç

Program anlama literatüründe programın amacı, hedefleri ve yazanın niyeti önemli kavramlardır; yukarıdan aşağı modellerde amaç hakkında hipotez kurmak sürecin başlangıç noktasıdır. Bu, dört kutunun cevaplamadığı bir soruyu görünür kılar: sistem bunu neden yapmaktadır.

Amacın ayrı bir boyut sayılmaması yönündeki gerekçe, bu sorunun cevabının çoğu zaman alan bilgisi, gereksinimler, iş hedefleri ve tasarım gerekçesinden gelmesidir. Bu bilgilerin bir kısmı evrim boyutunda zaten bulunabilir: bir kararın hangi gereksinim nedeniyle alındığı, o kararın tarihinde kayıtlıdır. Bu durumda amaç ayrı bir boyut olmaktan çıkar.

Amacın ayrı boyut sayılması için gösterilmesi gereken şey şudur: bir mühendisin davranış, yapı, evrim ve sınır hakkında yeterli bilgisi bulunmasına rağmen sistemi anlamasının hâlâ mümkün olmadığı ve eksik olanın amaç olduğu. Örnek olarak bir para birimi dönüştürme servisi verilebilir: davranışı, yapısı, ne zaman değiştiği ve hangi para birimlerini desteklediği bilinse dahi, bu dönüşümün neden o serviste yapıldığı bilinmiyorsa sistem anlaşılmış sayılmayabilir.

Kanıt

Amaçtan yapısal olarak farklı ikinci bir aday, kanıt ve doğrulamadır. İlk dört boyut sistem hakkında ne bilindiğini konu alır; bu boyut ise bilinenin nereden bilindiğini konu alır. Aradaki fark bir derece farkı değil, kategori farkıdır.

Regresyon testinin taşıdığı bilgi bu ayrımla netleşir. Test yalnızca geçmişte bir bozulma yaşandığını kaydetmez; o iddianın tekrar tekrar doğrulanabildiğini de gösterir. Bu okumada test kümesi sistemin bir parçası değil, sistem hakkındaki iddiaların kanıtıdır.

Çerçeve bu ayrımla şu hale gelir:

Ne biliyoruz?

Amaç	-> neden?
Davranış	-> ne yapıyor?
Yapı	-> nasıl organize?
Evrım	-> nasıl bu hale geldi?
Sınır	-> nerede geçerli?

Bildiğimizi nasıl biliyoruz?

Kanıt / doğrulama

Hatanın tekrar etmemesi problemi açısından bakıldığında, kanıt boyutu amaçtan daha merkezidir. Benzetmeye döndürüldüğünde bu şu düzeltmeyi verir: kod sistemin kendisi hakkında yazılmış bir temsildir, testler ise bu temsil hakkında sahip olunan kanıtların izleridir.

Sayfanın Dörde Bölünmesi

Dört boyut belirlendikten sonra benzetme genişletilebilir. Beyaz sayfa artık bölünmemiş bir yüzey değil, dört bölgeye ayrılmış bir alandır:

DAVRANIŞ Sistem ne yapıyor?	YAPI Nasıl organize?
EVRİM Nasıl bu hale geldi?	SINIR Nereye kadar?

Kod yazmak, test yazmak, hata gidermek, kullanımda gözlem yapmak ve yeniden düzenlemek gibi faaliyetlerin tümü bu sayfanın üzerine yeni izler bırakır. Başlangıçta sayfa boştur ve sistem hakkında az şey bilinmektedir. Çalışıldıkça izler artar. Olgun bir sistemde sayfa büyük ölçüde kararmıştır, ancak bu kodun karmaşıklaştığı anlamına gelmez; sistemin davranışı, yapısı, evrimi ve sınırları hakkında çok daha fazla şey bilindiği anlamına gelir.

Buradan çıkan formülasyon şudur: iyi mühendislik sayfayı temiz tutmak değil, sistem hakkındaki belirsizliği sistematik olarak azaltmaktır.

Anlamanın Ölçülmesi

Sistem anlayışı doğrudan gözlemlenebilen bir değişken değildir. Ölçüm, doğrudan bir değer atamakla değil anlamının gözlemlenebilir izleri üzerinden yapılmak zorundadır. Kavram bu haliyle gizil (*latent*) bir değişkendir.

Boyut başına gösterge önerileri

Davranış. Sistemin ne yapması gerektiğinin ne kadar bilindiği. Ölçülebilecekler: test edilen davranışların oranı, gereksinim ile test arasındaki eşleşme, kritik davranışların belirtimi, tanımsız davranış sayısı, kullanımda keşfedilen beklenmedik davranışlar.

Yapı. Sistemin nasıl oluştuğunun ne kadar bilindiği. Ölçülebilecekler: bağımlılık grafiğinin bilinen oranı, bileşen sahipliğinin belirli olup olmaması, döngüsel bağımlılıklar, mimari ihlaller, bileşen etkileşim testleri, bilinmeyen bağımlılıklar.

Evrım. Sistemin neden şu anda böyle olduğunun ne kadar bilindiği. Ölçülebilecekler: kritik kodun değişim geçmişi, hata ile değişiklik ve test arasındaki bağ, mimari kararların kayıtlı olup olmaması, kodun neden böyle olduğuna dair açıklamanın varlığı, regresyon testlerinin mevcudiyeti. Buradan türeyen özgün bir ölçü, bilgi korunumudur: ekip değiştiğinde sistem hakkındaki bilgi kaybolmakta mıdır.

Sınır. Sistemin nerede çalışıp nerede çalışmadığının ne kadar bilindiği. Ölçülebilecekler: sınır durumu kapsamı, arıza kipleri, girdi alanı kapsamı, hata yolu kapsamı, kullanımda ortaya çıkan olayların önceden bilinip bilinmediği. Bilinmeyen arıza kiplerinin sayısı özellikle değerli bir işarettir.

Birleştirmenin sakıncası

Dört boyutun basit bir ortalamayla tek bir sayıya indirgenmesi mümkündür, ancak yapılmamalıdır. Her boyutta seksen değerine ulaşılmış olması sistemin yüzde seksen anlaşıldığı anlamına gelmez; böyle bir yorumun dayanağı bulunmamaktadır. Doğru sıra, önce her boyutta gözlemlenebilir göstergelerin tanımlanması, birleştirmenin ise ancak bu göstergeler yerleştikten sonra düşünülmesidir.

Kapsam ile anlamamanın eşitlenemezliđi

Ölçüm tartışmasının merkezî bulgusu şudur: kapsam anlamamanın göstergesi değildir. Bir kodun tam kapsama sahip olması, o kodu kimsenin anlamadığı durumu dışlamaz.

Örnek, bir fiyata sabit bir katsayı uygulayan tek satırlık bir işlemdir. Bu işlev için tam kapsam elde etmek kolaydır. Ancak katsayının neden o değeri olduğu bilinmiyorsa, sistemin evrim ve alan bilgisi bilinmiyor demektir. Test, kodun çalıştığını gösterir; kodun neden öyle olduğunu göstermez.

Bu nedenle ölçülmesi gereken büyüklük çalıştırılan satır oranı değil, ortadan kaldırılan belirsizlik miktarıdır. Benzetmenin diliyle beyazlık bilinmeyene, karalama kanıtı karşılık gelir. Buradan çıkan teorik ifade şudur:

$$\text{Sistem anlayışı} \approx 1 - \text{Sistem belirsizliđi}$$

Bu formülasyonun değeri hedefi değıştirmesindedir. Soru artık kodun ne kadarının test edildiđi değil, sistem hakkındaki bilinmeyen alanın ne kadar küçültüldüğüdür.

Hatanın Tekrar Etmemesi

Çerçeve bırakılıp soru doğrudan ele alındığında ilk cevap regresyonun önlenmesidir. Ancak test yazmak tek başına yeterli değildir; mekanizmanın tamamı gereklidir.

Asgari mekanizma

1. Hatanın belirlenimci biçimde yeniden üretilmesi. Bir yerde bozulduğunun bilinmesi yetmez; hangi girdinin hangi koşulda hangi yanlış sonucu ürettiği belirlenmelidir.
2. Kök sebebin bulunması. Belirtinin bastırılması bu adımın yerine geçmez.
3. Düzeltmenin yapılması.
4. Hatayı otomatik olarak yakalayacak regresyon testinin eklenmesi.
5. Testin sürekli tümleştirme hattına bağlanması. Yalnızca geliştiricinin makinesinde duran bir test zayıf koruma sağlar.
6. Testin zorunlu kalite kapısı haline getirilmesi. Test başarısızsa birleştirmenin engellenmesi gerekir.

Bu mekanizma tamamlandığında test kurumsal hafızaya dönüşür: bir yıl sonra kodu değiştiren başka bir mühendis aynı hatayı geri getirdiğinde, test bunu yakalar.

Hata ile hata sınıfının ayrılması

Aynı hatanın tekrar etmemesi ile aynı türden hatanın tekrar etmemesi farklı hedeflerdir. Negatif bir indirim değerinin yanlış işlenmesi için yazılan bir test, aynı yapıdaki bir vergi hesabında ortaya çıkacak aynı türden hatayı yakalamaz.

Bu ayrım, sorunun yeniden formüle edilmesini gerektirir: bu belirli hatanın nasıl engelleneceği yerine, bu hataya izin veren sınıfın nasıl ortadan kaldırılacağı sorulur. Bu düzeyde devreye giren araçlar farklıdır: daha iyi soyutlama, değişmezler, tip sistemi, doğrulama, sözleşme testi, özellik tabanlı test, statik analiz ve mimari kısıtlar.

Olgunluk seviyeleri

1. Hata bulunur ve giderilir.
2. Hata bulunur, giderilir ve regresyon testi yazılır.
3. Hata sınıfı belirlenir ve sistem o sınıfı üretemeyecek hale getirilir.
4. Aynı arıza kipinin sistemin başka yerlerinde ortaya çıkması da engellenir.

Benzetmenin mühendislik karşılığı burada tamamlanır: her hata sistem hakkında yeni bir bilgi üretir ve iyi mühendislik bu bilgiyi bir sonraki değişiklikte unutulmayacak hale getirir. En basit mekanizma regresyon testidir; daha ileri mekanizma, hatayı mümkün kılan koşulun sistemden kaldırılmasıdır.

Lokal Çözümün Küresel Bozulma Üretmesi

Somut olay

Bir yapay zeka ajanı bir hatayı gidermiş, bu düzeltme beklenmedik bir yerde ikinci bir hata üretmiştir. Ajan gerekçesini basite kaçtığı biçiminde ifade etmiştir. Teknik karşılığı şudur: mevcut problem çözülürken sistemin geri kalanındaki davranışlar yeterince hesaba katılmamıştır.

Olayın yapısı şöyledir:

Hata A

- > düzeltme A
- > düzeltme A sistemin başka bir varsayımını bozdu
- > Hata B

İkinci hatanın beklenmemesinin nedeni, zihinsel sistem modelinin o bağlantıyı içermemesidir. Bu, test eksikliğinden farklı bir problemdir: hata A için yazılan regresyon testi hata A'nın tekrarını engeller, hata B'nin doğmasını engellemez.

Asıl problem şudur: bir değişikliğin sistemin başka hangi davranışlarını etkileyebileceği bilinmemektedir.

Savunma katmanları

- Regresyon testi: bu belirli hata tekrar çıkmasın.
- Entegrasyon ve sistem testleri: düzeltme başka parçaları bozmasın.
- Mevcut test kümesi: bilinmeyen başka davranışlar kırılırsa yakalansın.
- Değişiklik etki analizi: bu değişiklik sistemin neresine dokunuyor.
- Değişmezler ve sözleşmeler: hangi koşullar hiçbir değişiklikte bozulmamalı.
- Kod incelemesi ve gerekçelendirme: bu çözüm neden doğru, hangi varsayımlara dayanıyor.

Küresel düşünmeye geçiş

Geçiş, daha çok kod okumakla değil değişikliğin sistem üzerindeki etkisini modellemeyi zorunlu kılmakla sağlanır. Süreç şu adımlardan oluşur.

Birinci adım: problemi izole etmek. İlk refleks bir satırı değiştirmek olmamalıdır. Önce hatanın yeniden üretimi, kök sebebi ve etkilediği davranış belirlenir. Sorulan soru hatanın nerede olduğu değil, hangi davranışın sonucu olduğudur.

İkinci adım: değişiklik alanını çıkarmak. Değişen koddan başlanarak doğrudan çağırانlar, dolaylı çağırانlar, paylaşılan durum ve veri, dış bağımlılıklar ve testler sırayla çıkarılır. Bu, değişiklik etki analizinin kendisidir.

Örnek olarak bir fiyat hesaplama işlevi ele alınabilir:

```
calculatePrice()
    |
    +-----+
    |       |       |
    |       |       |
Sepet Fatura Ödeme
    |               |
    v               v
İndirim   Tahsilat
```

İşlev geliştiriciye küçük görünebilir; sistem açısından küçük olmayabilir.

Üçüncü adım: değişmezleri çıkarmak. Bu değişiklikten sonra kesinlikle doğru kalması gereken koşullar listelenir: fiyatın negatif olmaması, sipariş toplamının kalemlerin toplamına eşit olması, tahsilat tutarının sipariş toplamına eşit olması, indirim fiyatı aşmaması. Bunlar test değil sistem kurallarıdır; bir değişiklik bunlardan birini bozuyorsa lokal olarak çalışıyor olsa dahi küresel olarak yanlıştır.

Dördüncü adım: mevcut testleri etki haritası olarak kullanmak. Test kümesine yalnızca hatayı yakalayıp yakalamadığı sorulmaz; değiştirilen kodun hangi davranışlarını başka testlerin koruduğu sorulur. Bu okumada test kümesi sistemin bağlantı haritasını da verir.

Beşinci adım: değişikliği yapmak. Burada belirleyici ilke şudur: en küçük kod değişikliği en küçük sistem riski değildir. İki satırlık bir değişiklik gizli bir varsayımı bozarak yeni bir hata üretebilirken, on satırlık bir değişiklik daha güvenli olabilir. Optimize edilecek büyüklük değişikliğin boyutu değil sistem riskidir.

Altıncı adım: etki alanını doğrulamak. Testlerin piramit sırasına göre değil, değişikliğin etkileyebileceği davranışlara göre koşturulması gerekir. Bu yaklaşım risk temelli regresyon testi olarak adlandırılabilir.

Tek cümleyle ifade edildiğinde: lokal optimizasyon bu problemi nasıl çözerim sorusudur; küresel optimizasyon bu problemi çözerken sistemin geri kalanını nasıl bozmam sorusudur. Daha kesin biçimde: lokal yaklaşım düzeltmeyi, küresel yaklaşım düzeltmeden sonraki sistem durumunu optimize eder.

Ajan için süreç kurgusu

Bu ayrım özellikle yapay zeka kodlama ajanları için önemlidir, çünkü ajanın doğal eğilimi lokaldir: kendisine verilen kaydı mümkün olduğunca hızlı kapatmaktır. İstenen davranış ise sistem çapında akıl yürütmedir.

Ajanın izlemesi istenen akış şudur:

değişiklik

- > etki analizi (davranış, yapı, bağımlılıklar)
- > risk analizi
- > gerekli testlerin belirlenmesi
- > yamanın üretilmesi
- > regresyon kümesinin oluşturulması
- > etkinin doğrulanması

Yani ajandan istenen şey kod değişikliği üretmek değil, bu boru hattını takip etmektir.

Literatürdeki Karşılık

Değişiklik etki analizi

Yukarıda tarif edilen ikinci adımın literatürdeki adı değişiklik etki analizidir (*change impact analysis*). Bir değişikliğin yaratabileceği dalga etkisinin ve etkilenen öğelerin, değişiklik yapılmadan önce belirlenmesidir (Bohner ve Arnold, 1996). İki klasik yaklaşımı vardır: bağımlılık analizi kod öğeleri arasındaki bağları izler; izlenebilirlik analizi gereksinim, tasarım, kod ve test arasındaki ilişkileri izler.

Önemli bir ayrım kaydedilmelidir. Değişiklik etki analizi bir sıralama önermez; yalnızca etkilerin değişikliktan önce belirlenmesini söyler. Notta tarif edilen yöntem, bunun üzerine en az yan etkili değişiklik ve regresyon doğrulaması fikirlerini ekler. Dolayısıyla yöntemin tamamının tek bir yerleşik adı yoktur ve yerleşik terimlerin birleşimi olarak ifade edilmelidir:

kök sebep analizi

```
-> değişiklik etki analizi  
-> en az yan etkili değişiklik  
-> regresyon doğrulaması
```

Ayrıca lokal akıl yürütme (*local reasoning*) teriminin program doğrulama literatüründe yerleşik ve farklı bir anlamı bulunmaktadır: bir program parçası hakkında tüm küresel durumu modellemekten akıl yürütmek. Bu nedenle lokaldan küresele geçiş ifadesi, burada tarif edilen yöntemin standart adı olarak kullanılmamalıdır.

Alanın konumu

Değişiklik etki analizi yazılım bakımı ve yazılım evrimi alanının altında yer alır. Kabaca hiyerarşi şöyledir:

Yazılım Mühendisliği

Yazılım Bakımı

Değişiklik Yönetimi

Değişiklik Analizi

Değişiklik Etki Analizi

Program Anlama

Regresyon Testi

Hata Ayıklama

Yeniden Düzenleme

Yeniden Mühendislik

Terminolojide bir belirsizlik bulunmaktadır: yazılım evrimi daha geniş bir alan olarak da kullanılabilmekte ve bakım onun altında ya da onunla iç içe ele alınabilmektedir.

Komsu başlıkların tanımları şöyledir. **Program anlama**, mevcut programın ne yaptığını ve neden öyle yaptığını çıkarma faaliyetidir. **Hata ayıklama**, belirli bir yanlış davranışın kaynağının bulunmasıdır. **Regresyon testi**, değişiklikten sonra eski davranışların korunduğunun doğrulanmasıdır. **Yeniden düzenleme**, gözlenebilir davranış değiştirmeden iç yapının iyileştirilmesidir. **Yeniden mühendislik**, sistemin bütününe ya da büyük bir parçasının elden geçirilmesidir.

Bir hata giderme sürecinde bunlar şu zincirde birbirine bağlanır: hatanın lokal olarak düzeltilmesi değişikliktir, bu değişikliğin nereleri etkileyebileceğinin araştırılması etki analizidir, etkilenen yerlerin sınanması regresyon testidir.

Ele Alınmayan Konular

Konuşmanın pratik hedefi hatanın tekrar etmemesi olmasına rağmen, bu hedefe doğrudan bağlı birçok konu ele alınmamıştır. Bunların kaydedilmesi, çerçevenin nerede eksik olduğunun görülmesini sağlar.

Hata türlerinin sınıflandırılmaması

Tartışma hataların türü incelenmeden yürütülmüştür. Ortaya çıkan hataların sınır değeri hatası mı, eşzamanlılık hatası mı, yanlış anlaşılmış gereksinim mi yoksa entegrasyon hatası mı olduğu sorulmamıştır. Bu türlerin her birinin çözümü farklıdır.

En ucuz ilk adım bu sınıflandırmadır: yakın geçmişteki hataların kök sebebe göre gruplanması. Böyle bir gruplamada birkaç kategorinin toplamın büyük bölümünü oluşturması beklenir; bu durumda çabanın nereye yöneltileceği veriden okunur.

Tasarım tarafının konuşulmaması

Tartışmanın tamamı hata ortaya çıktıktan sonra ne yapılacağı ekseninde kalmıştır. Oysa bir işlev değiştiğinde beş ayrı yerin bozulması bir tasarım sorunudur. Değişiklik etki analizi bunu tespit eder, gidermez.

Etki alanını daraltan kararlar ayrı bir başlıktır: bağlılığın azaltılması, sınırların netleştirilmesi, geçersiz durumların temsil edilemez kılınması, yan etkinin ayrılması, durum sayısının azaltılması. Etki alanı küçükse analiz edilecek şey de az kalır. Bu ilkenin kuramsal temeli, modüllere ayırma ölçütünün işlem adımları değil gizlenen kararlar olması gerektiği önermesidir (Parnas, 1972).

Test dışındaki doğrulama araçlarının yok sayılması

Tartışma testi tek kanıt aracı olarak almıştır. Aynı işi daha ucuza yapan katmanlar mevcuttur: tip sistemi, statik analiz, biçim denetleyicileri, sözleşme ve değişmez kontrolleri, özellik tabanlı test. Bir hatayı derleme zamanında imkansız kılmak, çalışma zamanında sınamaktan hem ucuz hem kesindir.

Bu katmanlar bir maliyet sırası oluşturur; soldaki bir katmanda yakalanan hatanın maliyeti sağdakine göre kat kat düşüktür:

tip sistemi -> statik analiz -> değişmez -> test -> üretimde izleme

Benzetmenin diliyle, sayfayı karalamak yalnızca test yazmak değildir.

Üretim tarafının hiç ele alınmaması

Hiçbir hatanın çıkmaması ulaşılabilir bir hedef değildir. Ulaşılabilir hedef, hatanın erken yakalanması ve etkisinin sınırlanmasıdır. Bunun araçları ayrı bir ailedir: kayıt tutma, metrik, hata takibi, yetenek bayrağı, kademeli yayın ve hızlı geri alma. Tartışma tamamen yayın öncesi tarafta kalmış, yayın sonrası taraf hiç ele alınmamıştır.

Maliyetin tartılmaması

Her hatanın önlenmesinin bir bedeli vardır ve tartışma tüm hatalar eşit önemdeymiş gibi ilerlemiştir. Ödeme akışındaki bir hata ile bir raporlama ekranındaki biçimsel bir hata aynı yatırımı gerektirmez. Nereye ne kadar doğrulama yapılacağı sorusu sorulmamıştır.

Ölçüm sorusunun cevapsız kalması

Kodun anlaşılmasının sayısal olarak nasıl ölçüleceği sorusu sorulmuş, cevaplanmadan başka konuya geçilmiştir. Doğrudan ölçüm mümkün değildir; ancak dolaylı göstergeler mevcuttur: doğrulama katmanlarından kaçarak kullanıma ulaşan hata oranı, bir düzeltmenin yeni hata doğurma oranı, aynı kusurun tekrar açılma oranı, değişikliklerin başarısızlık oranı.

Bu göstergelerin işlevi anlayışın düzeyini bildirmek değil, anlayışın nerede yetersiz olduğunu göstermektir. Pratik değeri de buradan gelir.

Ajan örneğinde teşhisin eksik kalması

Basite kaçma ifadesi bir tutum açıklaması olarak alınmış ve çözüm bir talimata bağlanmıştır. Oysa daha olası açıklama bağlam eksikliğidir. Bir ajan, değiştirdiği işlevin nerelerden çağrıldığını görmüyorsa, küresel etkiyi bilmesi ilkel olarak mümkün değildir.

Bu teşhis çözümü de değiştirir. Çözüm bir talimat değil bir süreçtir: değişiklikten önce çağrı zincirinin, etkilenen testlerin ve ilgili sözleşmelerin ajana açıkça sunulması, küresel düşünmesini isteyen bir yönergeden daha etkilidir.

Benzetmenin sınanmaması

Sayfanın kararmasının iyi olduğu fikri tartışma boyunca sınanmadan kabul edilmiştir. Oysa karalama birikimi aynı zamanda bir yüküdür.

Kararsız testler kümenin güvenilirliğini düşürür ve bir eşikten sonra ekibin başarısız testleri ciddiye almamasına yol açar. Uygulama ayrıntısına bağlanmış testler her yapısal düzenlemede kırılır, buna karşılık gerçek hata bulmaz; bu testler yeniden düzenlemeyi caydırarak tasarımın iyileştirilmesini engeller. Bir noktadan sonra ekip test yazmaktan değil kodu değiştirmekten kaçınmaya başlar.

Dolayısıyla ölçüt sayfanın ne kadar karardığı değil, karalamanın ne kadarının gerçek bilgi taşıdığıdır. Buradan çıkan ve kolayca gözden kaçan sonuç şudur: test kümesinden test silmek de bir bakım faaliyetidir.

Yöntem Üzerine Bir Gözlem

Konuşmanın önemli bir bölümü boyut sayısının dört mü beş mi olduğu tartışmasında geçmiş, ardından çerçeve bir kenara bırakılıp asıl soruya dönülmüştür. Bu, iki nedenin birleşiminden doğmuştur.

Birincisi, sohbete katılan modelin çerçeve üretmeye fazla istekli olmasıdır. Sorulan her soru için yeni bir taksonomi önerilmiş, literatürdeki yerleşik terimlerle türetilen adlar bir noktada birbirine karışmıştır. Bu karışıklık, açıkça uyarıldıktan sonra düzeltilmiştir.

İkincisi, sorunun somut hedefinin baştan söylenmemiş olmasıdır. Konuşmayı başlatan asıl neden yazılan kodda hata tekrarının önlenmesiydi; bu ancak yirmi dakika sonra ifade edilmiştir. Hedef baştan bilinseydi, boyut sayısı tartışmasının bu hedefe katkısının sınırlı olduğu erken görülebilirdi.

Buradan çıkan genel kural şudur: kavramsal bir çerçeve arayışı, çerçevenin hangi somut kararı değiştireceği belirtilmeden yürütüldüğünde kendi kendini besler. Çerçevenin sınanma ölçütü, güzel görünmesi değil bir karara girdi sağlamasıdır.

Sonuç

Notta iki ayrı düzey bulunmaktadır ve bunların birbirine karıştırılmaması gerekir. Birincisi kavramsaldir: sistem hakkındaki bilginin dört boyutta toplanabileceği hipotezi. İkincisi pratiktir: bir düzeltmenin başka bir yeri bozmasını engelleyen süreç. İkinci düzey birinciden türememektedir ve birincinin doğruluğuna bağlı değildir.

Kavramsal düzeyde ulaşılan en dürüst konum, dört boyutun her birinin literatürde bağımsız karşılığı bulunduğu, ancak dördünün tek bir asgari küme olarak birleştirilmesinin bu çalışmaya ait bir iddia olduğudur. Bu iddianın sınanma ölçütü açıktır: bir boyut çıkarıldığında hangi soruların cevapsız kaldığı. Beşinci boyut adaylarından amaç, cevabının büyük ölçüde evrim boyutundan türetilmesi nedeniyle zayıftır; kanıt boyutu ise kategori olarak farklı olduğu için diğerlerinin yanına değil altına yerleşir.

Pratik düzeyde ulaşılan sonuç, düzeltmenin doğruluğunun düzeltilen davranışla değil sistem genelinde bıraktığı izle ölçüldüğüdür. Buradan çıkan sıralama analiz, seçenek üretimi ve seçim biçimindedir; bu sıranın bozulması, etki bilmeden üretilen seçeneklerin hepsinin lokal kalmasına yol açar.

Çerçevenin sınırlılıkları üç noktada toplanır. Birincisi, tartışmanın tamamı hata sonrası tarafta kalmıştır; hatayı baştan imkansız kılan katmanlar ve etki alanını daraltan tasarım kararları ayrı ve daha temel bir düzeydir. İkincisi, üretim tarafındaki gözlemlenebilirlik ve geri alma mekanizmaları hiç ele alınmamıştır; oysa gerçekçi hedef hatanın hiç çıkmaması değil etkisinin sınırlı kalmasıdır. Üçüncüsü, doğrulama yatırımının maliyeti hiç tartışılmamış, tüm hatalar eşit önemdeymiş gibi ilerlenmiştir.

Açık kalan asıl soru ölçümdür. Notta önerilen süreçlerin hata oranını filen düşürüp düşürmediği bilinmemektedir ve bu, ölçülmediği sürece bilinmez. Ölçümün yolu mevcuttur: geçmiş hataların kök sebebe göre gruplanması ve süreç öncesi ile sonrası düzeltmelerin yeni hata doğurma oranlarının karşılaştırılması. Bu ölçüm yapılana kadar notta yazılı olan her şey, kendi çerçevesinin diliyle, kanıt değil bilgidir.

Kaynaklar

- Bohner, S. A., & Arnold, R. S. (1996). *Software Change Impact Analysis*. IEEE Computer Society Press.
- Brooks, R. (1983). Towards a Theory of the Comprehension of Computer Programs. *International Journal of Man-Machine Studies*, 18(6).
- Détienne, F. (2002). *Software Design: Cognitive Aspects*. Springer.
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.

- Girba, T., & Ducasse, S. (2006). Modeling History to Analyze Software Evolution. *Journal of Software Maintenance and Evolution*, 18(3).
- INCOSE (2015). *Systems Engineering Handbook*, 4. baskı. Wiley.
- ISO/IEC 14764. *Software Engineering — Software Life Cycle Processes — Maintenance*.
- Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9).
- Mens, T., & Demeyer, S. (2008). *Software Evolution*. Springer.
- Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12).
- von Mayrhauser, A., & Vans, A. M. (1995). Program Comprehension During Software Maintenance and Evolution. *IEEE Computer*, 28(8).
- Weiser, M. (1981). Program Slicing. *ICSE*.
- Yin, Z., Yuan, D., Zhou, Y., Pasupathy, S., & Bairavasundaram, L. (2011). How Do Fixes Become Bugs? *ESEC/FSE*.